

Privacy Cash EVM security review

Intro

This security review was conducted by kriko.eth. The subject were the Privacy Cash Solidity contracts at commit `3a5acf7` and USDC upgrade at commit `1e9af6a`.

Risk Classification

The risk is assessed based on the severity matrix:

Likelihood		Impact	
	HIGH	MEDIUM	LOW
HIGH	CRITICAL	HIGH	MEDIUM
MEDIUM	HIGH	MEDIUM	LOW
LOW	MEDIUM	LOW	INFO

Findings Summary

Total issues discovered:

Severity	Count	Addressed
Critical	0	0
High	0	0
Medium	0	0
Low	3	3
Info	0	0

Engagement Summary

The subject of this review was the EVM implementation of the Privacy Cash protocol at commit `3a5acf7`. Several low-severity issues regarding gas griefing were identified and then fixed by the team, either on the contract or infrastructure level. A subsequent mitigation review at commit `f5a6758` was conducted with no further findings. Review of the USDC upgrade at commit `1e9af6a` was conducted with no additional issues identified.

Low Severity Findings

PCEVM-L01: Returnbomb gas griefing via malicious recipient in ETH transfers

Finding Description

`EtherPool.transact()` performs two ETH transfers using `.call{value:}` (lines 91 and 97). Under Solidity 0.8.24 with the legacy codegen pipeline, discarding the return data variable

does not prevent the compiler from emitting `RETURNDATACOPY`. The caller pays for memory expansion to hold the callee's return data regardless.

```
// contracts/EtherPool.sol, line 91
(bool success, ) = _extData.recipient.call{value:
↳ uint256(-_extData.extAmount)}("");

// contracts/EtherPool.sol, line 97
(bool success, ) = _extData.feeRecipient.call{value: _extData.fee}("");
```

A malicious `recipient` or `feeRecipient` contract can return a large payload from its `fallback()`, forcing the caller to pay for memory expansion. Neither address has a byte-code or return data size restriction. A 1 MB returnbomb inflates withdrawal gas from ~500K to ~2.7M. Both addresses returning large payloads simultaneously can push gas past the 30M block limit due to the EVM's quadratic memory expansion cost, making the withdrawal uncludable.

The victim is the relayer submitting `transact()`. The attacker sets `recipient` / `feeRecipient` to returnbomb contracts and offers a fee that does not cover the inflated gas. In practice, relayers simulate transactions before broadcasting, which catches the inflated cost. Direct user submissions are self-harm only.

Recommendation

Replace `.call{value:}` with assembly calls that set the return data buffer to zero, avoiding `RETURNDATACOPY` entirely:

```
- (bool success, ) = _extData.recipient.call{value:
↳ uint256(-_extData.extAmount)}("");
- require(success, "ETH transfer failed");
+ bool success;
+ address recipient = _extData.recipient;
+ uint256 amount = uint256(-_extData.extAmount);
+ assembly {
+     success := call(gas(), recipient, amount, 0, 0, 0, 0)
+ }
+ require(success, "ETH transfer failed");
```

```
- (bool success, ) = _extData.feeRecipient.call{value: _extData.fee}("");
- require(success, "Fee transfer failed");
+ bool success;
+ address feeRecipient = _extData.feeRecipient;
+ uint256 feeAmount = _extData.fee;
+ assembly {
+     success := call(gas(), feeRecipient, feeAmount, 0, 0, 0, 0)
+ }
+ require(success, "Fee transfer failed");
```

Status

Acknowledged. The protocol will add a relayer simulation to bound the gas consumption.

PCEVM-L02: Unbounded `encryptedOutput` bytes enable event log bloat

Finding Description

The `encryptedOutput1` and `encryptedOutput2` fields in `ExtData` (`EtherPool.sol:29-30`) are unbounded `bytes` with no length check anywhere in `transact()`. The `extDataHash` integrity check at line 77 hashes the entire `ExtData` into a single field element, so the ZK circuit is agnostic to pre-image size.

Both fields are emitted directly in `NewCommitment` events at lines 102-103:

```
emit NewCommitment(_args.outputCommitments[0], nextIndex - 2,
  ↪ _extData.encryptedOutput1);
emit NewCommitment(_args.outputCommitments[1], nextIndex - 1,
  ↪ _extData.encryptedOutput2);
```

An attacker can submit valid transactions with arbitrarily large encrypted output blobs (e.g. 100 KB each). Legitimate encrypted outputs are a fixed, known size (typically 68-256 bytes). Off-chain infrastructure indexing `NewCommitment` events (relayers, wallets, RPC providers) is forced to download and process the oversized payloads. On L2 chains where calldata is 10-100x cheaper than L1, this can impose significant storage and bandwidth costs on off-chain systems. No on-chain state corruption or fund loss occurs.

Recommendation

```
+ uint256 public constant MAX_ENCRYPTED_OUTPUT_SIZE = 256;
+
+ function transact(Proof memory _args, ExtData memory _extData) public
  ↪ payable nonReentrant whenNotPaused {
+   require(_extData.encryptedOutput1.length <= MAX_ENCRYPTED_OUTPUT_SIZE,
  ↪ "Encrypted output 1 too large");
+   require(_extData.encryptedOutput2.length <= MAX_ENCRYPTED_OUTPUT_SIZE,
  ↪ "Encrypted output 2 too large");
   require(isKnownRoot(_args.root), "Invalid merkle root");
   require(!isSpent(_args.inputNullifiers[0]) &&
  ↪ !isSpent(_args.inputNullifiers[1]), "Input is already spent");
```

Set `MAX_ENCRYPTED_OUTPUT_SIZE` to the exact expected ciphertext size for the protocol's encryption scheme.

Status

Fixed.

PCEVM-L03: Missing minimum deposit amount enables dust deposit grieving

Finding Description

`EtherPool.sol` enforces a maximum deposit amount but no minimum, allowing deposits as small as 1 wei:

```
// EtherPool.sol:85-87
if (_extData.extAmount > 0) {
    require(msg.value == uint256(_extData.extAmount), "Incorrect ETH value");
    require(uint256(_extData.extAmount) <= maximumDepositAmount, "amount is
    ↪ larger than maximumDepositAmount");
}
```

Each deposit calls `_insert()` (`MerkleTreeWithHistory.sol:58`), which writes a new Merkle root into a fixed-size ring buffer of 100 slots (`ROOT_HISTORY_SIZE`, line 24):

```
// MerkleTreeWithHistory.sol:79-82
uint32 newRootIndex = (currentRootIndex + 1) % ROOT_HISTORY_SIZE;
currentRootIndex = newRootIndex;
roots[newRootIndex] = currentLevelHash;
nextIndex = _nextIndex + 2;
```

After 100 dust deposits, every pre-existing root is overwritten. `isKnownRoot()` then returns `false` for all prior roots, and any withdrawal referencing one reverts at line 75 with `"Invalid merkle root"`. Affected users must regenerate their ZK proofs against a current root and resubmit.

The ZK circuit disables the Merkle proof check for zero-value input UTXOs, so the attacker needs no existing UTXOs and can pre-compute all 100 proofs offline using the publicly readable root from `getLastRoot()`. No rate limiting exists in the contracts.

On L2 deployments, the full 100-deposit attack costs approximately 0.005 ETH per round and can execute within a single block. Sustained dust deposits (one per block) reduce the root validity window to approximately 20 minutes, creating a race condition between proof generation and root eviction. No direct fund loss is possible, but sustained grieving at under 0.02 ETH/hour can effectively deny withdrawals to all pool users.

Recommendation

Add a `minimumDepositAmount` state variable and enforce it alongside the existing maximum. Set the minimum high enough that locking 100x the minimum represents meaningful capital cost for an attacker.

In `EtherPool.sol`, add the state variable and enforce it during deposit validation:

```
- uint256 public maximumDepositAmount;
+ uint256 public minimumDepositAmount;
+ uint256 public maximumDepositAmount;
```

```

    if (_extData.extAmount > 0) {
        require(msg.value == uint256(_extData.extAmount), "Incorrect ETH
↪ value");
+   require(uint256(_extData.extAmount) >= minimumDepositAmount, "amount is
↪ smaller than minimumDepositAmount");
        require(uint256(_extData.extAmount) <= maximumDepositAmount, "amount is
↪ larger than maximumDepositAmount");
    }

```

Update `_configureLimits` to accept both bounds:

```

- function _configureLimits(uint256 _maximumDepositAmount) internal {
-   maximumDepositAmount = _maximumDepositAmount;
-   emit LimitsConfigured(_maximumDepositAmount);
- }
+ function _configureLimits(uint256 _minimumDepositAmount, uint256
↪ _maximumDepositAmount) internal {
+   require(_minimumDepositAmount <= _maximumDepositAmount, "min exceeds
↪ max");
+   minimumDepositAmount = _minimumDepositAmount;
+   maximumDepositAmount = _maximumDepositAmount;
+   emit LimitsConfigured(_minimumDepositAmount, _maximumDepositAmount);
+ }

```

As a complementary measure, consider increasing `ROOT_HISTORY_SIZE` in `MerkleTreeWithHistory.sol` (line 24) from 100 to a larger value such as 1000, which proportionally increases the cost to evict all roots.

Status

Fixed.